Lab 2 Class Task: Arrays, Binary Search and Sorting

Data Structures Lab (CSDS-224L)

Name: Zahra Akbar Roll No: 9250103 Date: 21/10/26

Instructions

- Complete the tasks in order. Save each one as its own file: `task1.cpp`, `task2.cpp`, and so on.
- Type the code yourself. Run it and check that your output matches the sample run.
- Show each working task to your instructor and get it signed in the checklist at the end.
- Functions go **above** `main`. Calls go **inside** `main`.
- Binary search only works on a **sorted** array.

Task 1: Iterative binary search (ages)

The ages of 8 students are stored in ascending order:

```
12 15 18 21 25 30 34 41
```

Write a function that uses **iterative** binary search to find an age. In `main`, ask the user to enter an age, call the function, and print the index where it was found, or a message that it was not found.

Hint: start with this function header.

```
int binarySearch(int arr[], int size, int key)
```

Sample runs:

```
Enter age to search: 25
25 found at index 4

Enter age to search: 20
20 not found
```

```cpp
#include <iostream>
using namespace std;
int binarySearch(int arr[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    while (low <= high)
    {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
int main()
{
    int arr[8] = {12, 15, 18, 21, 25, 30, 34, 41};
    int age;
    cout << "Enter age to search: ";
    cin >> age;
    int result = binarySearch(arr, 8, age);
    if (result == -1)
        cout << age << " not found";
    else
        cout << age << " found at index " << result;
    return 0;
}
```

Task 2: Recursive binary search (prices)

The prices of 7 items are stored in ascending order:

`50 120 175 240 300 450 620`

Write a **recursive** binary search function. Ask the user for a price and print the index or a not-found message, in the same way as Task 1.

Hint: you need two base cases. Do not forget the `return` before each recursive call.

```
int binarySearch(int arr[], int low, int high, int key)
```

Sample runs:

```
Enter price to search: 300
300 found at index 4
Enter price to search: 100
100 not found
```

```cpp
#include <iostream>
using namespace std;

int binarySearch(int arr[], int low, int high, int key)
{
    if (low > high)
        return -1;
    int mid = (low + high) / 2;
    if (arr[mid] == key)
        return mid;
    if (arr[mid] < key)
        return binarySearch(arr, mid + 1, high, key);
    return binarySearch(arr, low, mid - 1, key);
}

int main()
{
    int arr[7] = {50, 120, 175, 240, 300, 450, 620};
    int price;

    cout << "Enter price to search: ";
    cin >> price;

    int result = binarySearch(arr, 0, 6, price);

    if (result == -1)
        cout << price << " not found";
    else
        cout << price << " found at index " << result;

    return 0;
}
```

Task 3: Selection sort (read and sort)

Read 6 integers from the keyboard into an array. Sort them in **ascending** order using **selection sort**. Print the sorted array, then print the smallest and the largest value.

Hint: after sorting, the smallest value is `arr[0]` and the largest is `arr[size - 1]`.

```cpp
void selectionSort(int arr[], int size)
```

```cpp
#include <iostream>
using namespace std;

void selectionSort(int arr[], int size)
{
    for (int i = 0; i < size - 1; i++)
    {
        int min = i;
        for (int j = i + 1; j < size; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}
int main()
{
    int arr[6];
    cout << "Enter 6 numbers: ";
    for (int i = 0; i < 6; i++)
        cin >> arr[i];
    selectionSort(arr, 6);
    cout << "Sorted: ";
    for (int i = 0; i < 6; i++)
        cout << arr[i] << " ";
    cout << "\nSmallest: " << arr[0];
    cout << "\nLargest: " << arr[5];
    return 0;
}
```

Task 4: Bubble sort with a pass counter

Write a bubble sort (ascending) that uses a `swapped` flag to stop early, and make the function **return the number of passes** it made (count the last pass that finds nothing to swap). Test it on two arrays:

- `9 7 5 3 1` (reverse order)
- `1 2 3 4 5` (already sorted)

Hint: add one to `passes` at the start of every pass. Why does the already sorted array need fewer passes?

```
int bubbleSort(int arr[], int size)
```

Expected output:

```
Array 1 sorted: 1 3 5 7 9   Passes: 4
Array 2 sorted: 1 2 3 4 5   Passes: 1
```

```cpp
#include <iostream>
using namespace std;
int bubbleSort(int arr[], int size)
{
    int passes = 0;
    for (int i = 0; i < size - 1; i++)
    {       passes++;
        bool swapped = false;
        for (int j = 0; j < size - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            } }
        if (!swapped)
            break;
    return passes;
int main()
{   int arr1[5] = {9, 7, 5, 3, 1};
    int arr2[5] = {1, 2, 3, 4, 5};
    int p1 = bubbleSort(arr1, 5);
    int p2 = bubbleSort(arr2, 5);
    cout << "Array 1 sorted: ";
    for (int i = 0; i < 5; i++)
        cout << arr1[i] << " ";
    cout << " Passes: " << p1 << endl;
    cout << "Array 2 sorted: ";
    for (int i = 0; i < 5; i++)
        cout << arr2[i] << " ";
    cout << " Passes: " << p2 << endl;
```

Task 5: Insertion sort with a shift counter

Sort the array `23 11 45 8 30` in ascending order using **insertion sort**. Make the function return the **total number of shifts**, which is the number of times a value is moved one place to the right inside the `while` loop.

Hint: add one to `shifts` each time you run `arr[j + 1] = arr[j];`.

```
int insertionSort(int arr[], int size)
```

Expected output:

```
Sorted: 8 11 23 30 45
Shifts: 5
```

```cpp
#include <iostream>
using namespace std;
int insertionSort(int arr[], int size)
{
    int shifts = 0;
    for (int i = 1; i < size; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            shifts++;
            j--;
        }
        arr[j + 1] = key;
    }
    return shifts;
}
int main()
{
    int arr[5] = {23, 11, 45, 8, 30};
    int shifts = insertionSort(arr, 5);
    cout << "Sorted: ";
    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    cout << "\nShifts: " << shifts;
    return 0;
}
```

Task 6: Sort first, then search

Combine what you have learned. Read 8 integers from the keyboard (in any order). Sort them with **any** sorting function from Tasks 3 to 5. Print the sorted array. Then ask the user for a value and search for it with **binary search** (iterative or recursive). Print its index in the sorted array, or a not-found message.

Then run it again and search for `40`. It should print `40 not found`.

Question to answer in your notebook: what goes wrong if you run binary search **before** sorting?

Sample run:

```
Enter 8 numbers: 52 17 83 24 9 61 38 70
Sorted: 9 17 24 38 52 61 70 83
Enter value to search: 61
61 found at index 5
```

```cpp
#include <iostream>
using namespace std;
void selectionSort(int arr[], int size)
{    for (int i = 0; i < size - 1; i++)
    {       int min = i;
        for (int j = i + 1; j < size; j++)
        {          if (arr[j] < arr[min])
                min = j;
        }
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    } }
int binarySearch(int arr[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    while (low <= high)
    {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    return -1;
int main()
{   int arr[8];
    int key;
    char choice;
    cout << "Enter 8 numbers: ";
    for (int i = 0; i < 8; i++)
```

```cpp
    for (int i = 0; i < 8; i++)
        cin >> arr[i];
    selectionSort(arr, 8);
    cout << "Sorted: ";
    for (int i = 0; i < 8; i++)
        cout << arr[i] << " ";
    do
    {   cout << "\nEnter value to search: ";
        cin >> key;
        int result = binarySearch(arr, 8, key);
        if (result == -1)
            cout << key << " not found";
        else
            cout << key << " found at index " << result;
        cout << "\nSearch again? (y/n): ";
        cin >> choice;
    } while (choice == 'y' || choice == 'Y');

    return 0;
}
```

Task 7 (Challenge): Binary search on a descending array

This array is sorted from **highest to lowest**:

`90 75 60 45 30 15`

The normal binary search will fail on it. Change **one comparison** so that it works for a descending array. Search for `45` and for `50`.

Expected output:

```
45 found at index 3
50 not found
```

```cpp
#include <iostream>
using namespace std;
int binarySearch(int arr[], int size, int key)
{
    int low = 0;
    int high = size - 1;
    while (low <= high)
    {  int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        if (arr[mid] < key)
            high = mid - 1;
        else
            low = mid + 1;
    return -1;
int main()
{
    int arr[6] = {90, 75, 60, 45, 30, 15};
    int key;
    cout << "Enter value to search: ";
    cin >> key;
    int result = binarySearch(arr, 6, key);
    if (result == -1)
        cout << key << " not found";
    else
        cout << key << " found at index " << result;
    return 0;
}
```